

VR System Architecture

Below is a **clear, structured, and detailed explanation** of the entire content you shared, strictly focusing on concepts, flow, and technical meaning, exactly as expected for **CGVR / VR Modeling / PAGP / Computer Graphics theory answers**.

1. VR SYSTEM ARCHITECTURE (Figure: VR System Architecture)

A **Virtual Reality system** is not just graphics; it is an integrated loop of **hardware, software, user interaction, and real-time computation**.

1.1 Core Components

(a) VR Engine

The VR engine is the **computational core** of the system. Its responsibilities include:

- Scene management
- Rendering pipeline execution
- Physics simulation
- Collision detection
- Object behavior updates
- Synchronization between input and output

It continuously processes input data and updates the virtual environment in real time.

(b) I/O Devices

Input/Output devices act as the **bridge between the real world and the virtual world**.

Input devices

- Data gloves
- Motion trackers
- HMD position sensors
- Joysticks, controllers

Output devices

- Head Mounted Displays (HMD)
- Stereo screens
- Haptic feedback devices
- Audio systems

These devices send **user actions** to the VR engine and receive **sensory feedback** from it.

(c) Software and Databases

This block contains:

- 3D object models
- Textures
- Animation data
- Physical properties
- Behavior scripts

These databases are accessed by the VR engine to populate and update the virtual world.

(d) User and Task

- The **user** interacts with the system through I/O devices.
- The **task** defines the goal (training, simulation, walkthrough, surgery, design, etc.).

The system forms a **closed real-time feedback loop**:

User → Input → VR Engine → Output → User

2. VR MODELING CYCLE (Figure 5.1 – VR Modeling Cycle)

This diagram explains how a virtual environment is **created, managed, and executed**.

2.1 VR Authoring Tool (Central Component)

A VR authoring tool integrates multiple modeling aspects:

(a) Geometric Modeling

Defines:

- Shape of objects
- Vertices, edges, polygons
- Mesh topology

(b) Kinematic Modeling

Defines:

- Object motion
 - Joint constraints
 - Degrees of freedom
- Example: rotation of a door, bending of an arm.

(c) Physical Modeling

Defines:

- Mass
- Inertia
- Gravity
- Collision response
- Hardness, elasticity

(d) Intelligent Behavior

Defines:

- Object logic
 - Autonomous behavior
 - Event-driven responses
- Example: doors opening automatically, NPC movement.

(e) I/O Mapping

Maps physical input devices to virtual actions.

Example:

- Glove → Virtual hand
- Tracker → Camera viewpoint

Requires **calibration and device coding**.

(f) Model Management

Optimizes models to ensure:

- Real-time performance
- Level-of-detail (LOD)
- Memory efficiency

2.2 Runtime Simulation

All modeled data flows into **runtime simulation**, where the VR engine executes everything in real time.

3. GEOMETRIC MODELING (Section 5.1)

Geometric modeling defines **what virtual objects look like**.

3.1 Object Shape Representation

3.1.1 Triangle Meshes (Figure 5.2a)

A triangle mesh represents a surface using:

- Vertices (x, y, z)
- Edges
- Triangles

Key Properties

- Adjacent triangles share vertices
- Example: shared vertices (X_1, Y_1, Z_1) between Triangle 1 and Triangle 2
- Reduces memory usage
- Faster rendering

Advantages

- Efficient storage
- Hardware optimized (MIMD architectures)
- Easy transformations
- Supports level-of-detail optimization

Limitations

- Poor for smooth curved surfaces
- Requires many triangles for fine detail

3.1.2 Parametric Surfaces (Figure 5.2b)

Used to represent **smooth and curved objects**.

A parametric curve is defined as:

- $x(t), y(t), z(t)$, where $t \in [0,1]$

Cubic spline equations:

- $x(t) = a_x t^3 + b_x t^2 + c_x t + d_x$
- $y(t) = a_y t^3 + b_y t^2 + c_y t + d_y$
- $z(t) = a_z t^3 + b_z t^2 + c_z t + d_z$

Parametric surfaces extend this using two parameters:

- $x(s, t), y(s, t), z(s, t)$

Advantages

- Very smooth surfaces
- Less storage
- Ideal for organic shapes

Disadvantages

- Hard to deform
 - Cannot directly manipulate surface vertices
 - Control points lie outside the surface
-

4. METHODS OF CREATING 3D OBJECTS

4.1 Using Toolkit Editors

- Manually define vertices and connectivity
 - Requires programming skill
 - Time-consuming
 - Suitable only for simple models
-

4.2 Importing CAD Files (AutoCAD Example – Figure 5.3)

CAD tools like AutoCAD and 3D Studio:

- Widely used in architecture and mechanical design
- Allow reuse of existing models

Process

1. Create model in CAD
2. Export as DXF
3. Convert to VR-compatible format
4. Animate components separately (doors, fans)

Limitations

- DXF does not preserve object hierarchy
 - Separate files needed for moving parts
-

4.3 Using 3D Digitizers

Example devices:

- Immersion MicroScribe

- Polhemus HyperSpace

Working

- Physical stylus traces real object
- Records (x, y, z) coordinates
- Creates polygon meshes

Advantages

- Good for objects not in databases
- Accurate shape capture

Limitations

- Time-consuming
 - Limited work volume
 - Hand tremor and noise (~1 mm accuracy)
-

4.4 Using 3D Scanners

Example:

- Polhemus FastScan
- Cyrax 2500 (long-range)

Working Principle

- Laser triangulation
- Measures distance using camera-laser geometry

Advantages

- Non-contact
- Fast
- Suitable for large objects

Output

- Dense point cloud (x, y, z, color)

Post-processing Steps (Figure 5.6)

1. Point cloud generation
2. Triangle mesh construction

3. Polygon decimation (up to 95%)
 4. Conversion to NURBS surfaces
 5. Texture mapping using captured images
-

4.5 Online 3D Object Databases

Commercial model libraries provide:

- Pre-built objects
- Multiple Levels of Detail (L, M, H)

Pros

- Fast development
- Ready-to-use assets

Cons

- Expensive
 - Limited customization
 - Fixed database content
-

5. SUMMARY OF VR MODELING

VR modeling is a **multi-stage, multidisciplinary process** involving:

- Hardware mapping
- Mathematical surface representation
- Geometry optimization
- Real-time constraints
- Human interaction

The goal is to achieve:

- High realism
 - Real-time performance
 - Accurate physical and behavioral simulation
-

If you want, I can next:

- Convert this into **exam-ready answers**
- Provide **diagrams with labeled explanations**
- Give **short vs long answer formats**

- Link this directly to **rendering pipeline & parallel architectures**

Just tell me.

Below is a **complete, exam-oriented, and technically detailed explanation** of **Section 5.1.2 – Object Visual Appearance**, covering **scene illumination, radiosity, and texture mapping**, strictly aligned with the figures you provided and written in the tone expected for **Computer Graphics / VR / CGVR / PAGP theory answers**.

5.1.2 OBJECT VISUAL APPEARANCE

After geometric modeling defines the **shape** of virtual objects, **visual appearance modeling** determines **how those objects look when rendered**. Visual appearance is primarily influenced by:

1. **Scene illumination** (how light interacts with surfaces)
2. **Surface texturing** (how fine details are added without increasing geometry)

Together, these determine **realism, depth perception, and scene complexity**.

5.1.2.1 SCENE ILLUMINATION

Scene illumination computes **light intensity values** on object surfaces so that objects become visible and visually meaningful.

Classification of Illumination Models

1. Local Illumination

- Treats each object independently
- Considers only:
 - Light sources
 - Surface reflectivity
 - Viewer position
- Ignores:
 - Shadows cast by other objects
 - Light reflected between objects

Local illumination is computationally efficient and suitable for **real-time VR**.

2. Global Illumination

- Models **interreflection of light between objects**

- Includes:
 - Soft shadows
 - Color bleeding
 - Indirect lighting
- Produces **photorealistic scenes**
- Computationally expensive

LOCAL ILLUMINATION TECHNIQUES

(a) Flat Shading

- One normal per polygon
- One intensity value for entire polygon
- Results in visible polygon boundaries
- Very fast but visually unrealistic

(b) Gouraud Shading (1971)

Gouraud shading improves smoothness by **interpolating light intensities** across polygon surfaces.

Steps Involved

1. Vertex Normal Computation

- Each vertex normal is computed by averaging normals of adjacent polygons

$$\vec{N}_v = \sum_i \vec{N}_i$$

2. Vertex Illumination

- Lighting equations (ambient, diffuse, specular) are evaluated at vertices

3. Edge Intensity Interpolation

- Intensities are interpolated along polygon edges:

$$I_a = I_1 - (I_1 - I_2) \frac{y_1 - y_s}{y_1 - y_2}$$

$$I_b = I_1 - (I_1 - I_3) \frac{y_1 - y_s}{y_1 - y_3}$$

4. Scanline Interpolation

- Pixel intensity is interpolated across scanlines:

$$I_p = I_b - (I_b - I_a) \frac{x_b - x_p}{x_b - x_a}$$

Advantages

- Smooth appearance
- Faster than Phong shading
- Suitable for real-time systems

Limitations

- Specular highlights may be missed
 - Accuracy depends on mesh density
-

(c) Phong Shading (1975)

Phong shading improves realism by **interpolating surface normals instead of intensities**.

Process

1. Vertex normals are interpolated along edges
2. Normals are interpolated across scanlines
3. Each pixel normal is normalized
4. Illumination model is applied **per pixel**

Advantages

- Accurate specular highlights
- Realistic shiny surfaces
- Superior to Gouraud shading

Disadvantages

- Computationally expensive
- Earlier GPUs could not support it in real time

Advances in GPU hardware and optimized algorithms have made real-time Phong shading feasible.

GLOBAL ILLUMINATION: RADIOSITY

Global illumination accounts for **energy exchange between surfaces**.

Radiosity Concept

- Every surface emits, reflects, and absorbs light
- Light energy is distributed across surfaces as **radiosity**
- Independent of viewer position

Radiosity Computation Steps

1. Divide scene into **small surface patches**
2. Compute energy exchange between all patches
3. Iterate until energy distribution stabilizes
4. Assign vertex colors to patches

Characteristics

- View-independent lighting
- Extremely realistic
- Highly computation-intensive

Traditionally computed **offline**, making it suitable only for static environments.

INTERACTIVE RADIOSITY WITH GRAPHICS PIPELINE (Figure 5.10)

The diagram shows how **radiosity is synchronized with a real-time rendering pipeline**.

Architecture Explanation

- **Main Graphics Pipeline**
 - Application stage
 - Geometry stage
 - Rasterizer stage
- **Radiosity Coprocessor**
 - Performs global illumination calculations
- **Shared Memory**
 - Transfers patch vertex colors between processors

Workflow

1. Geometry is loaded into both pipelines
2. Radiosity processor computes global illumination
3. Patch vertex colors are sent to rendering pipeline
4. User changes viewpoint → no recomputation needed
5. User changes geometry → partial radiosity update triggered
6. Scene coherence allows efficient updates

This approach enables **interactive VR with global illumination**, at the cost of slight update delays.

5.1.2.2 TEXTURE MAPPING

Texture mapping improves realism **without increasing polygon count**.

Definition

Texture mapping is a rasterization-stage technique that modifies surface properties such as:

- Color
- Reflectivity
- Surface normals

Texture Mapping Process (Figure 5.11)

1. Object vertices are stored in:
 - Object space (x, y, z)
 - Parametric space (s, t)
2. A **corresponder function** maps:
 - (s, t) → (u, v) texture coordinates
3. (u, v) indexes into a **texture array**
4. The corresponding **texel color** is retrieved
5. Texel color modulates shaded pixel color

$$Final\ Color = Surface\ Color \times Texel\ Color$$

Texture Creation Methods

1. Paint programs
2. Digitized photographs
3. Analytical functions (procedural textures)
4. Commercial texture databases

Textures are typically square (e.g., 512 × 512 texels).

Advantages of Texture Mapping

- Increased realism
- Better depth and spatial cues
- Significant reduction in polygon count
- Higher frame refresh rates

Example:

- Tree with geometry: ~45,000 polygons
 - Tree with texture: 1 polygon
-

MULTITEXTURING

Multitexturing applies **multiple textures sequentially** using blending cascades.

(a) Bump Mapping

- Uses grayscale texture representing height variation
- Modifies pixel normals
- Creates illusion of surface detail
- No additional polygons

Used for:

- Rough surfaces
 - Melted wax
 - Stone, skin, fabric
-

(b) Light Mapping

- Precomputed grayscale texture
- Encodes light intensity from a source
- Applied during rasterization

Advantages

- High-quality lighting
- Low polygon requirement

Limitation

- Static lighting
 - Requires precomputation for each light configuration
-

Normal Mapping and Modern GPUs

Modern GPUs support:

- Normal maps (texture storing normal vectors)
- Per-pixel dot-product lighting
- Dynamic lighting independent of light count

This enables:

- High realism
 - High performance
 - Dynamic scenes
-

SUMMARY

Object visual appearance modeling involves:

- **Local illumination** for speed
- **Global illumination** for realism
- **Radiosity** for energy-based lighting
- **Texture mapping** for detail without geometry
- **Multitexturing** for advanced surface effects

The goal is to balance:

- Visual realism
 - Computational cost
 - Real-time interactivity
-

If you want, I can now:

- Convert this into **short exam answers**
- Create **step-wise diagram explanations**
- Provide **numerical comparison tables**
- Link this to **GPU parallelism and pipeline stages**

Just say what you need next.

Below is a **full, step-by-step, conceptually deep explanation** of **Section 5.2 – Kinematics Modeling**, written exactly in the style expected for **Computer Graphics / VR / CGVR / PAGP examinations**, and tightly aligned with **Figures 5.15, 5.16, 5.18, and 5.19** you provided.

5.2 KINEMATICS MODELING

After defining **object shape (geometric modeling)** and **object appearance (illumination and texturing)**, the next essential component of virtual world modeling is **kinematics modeling**.

Kinematics modeling answers three fundamental questions:

1. **Where is an object located in the virtual world?**
2. **How does the object move over time?**
3. **How is the virtual world viewed by the user (camera motion and projection)?**

Kinematics modeling is purely concerned with **motion and spatial relationships**, not forces (those belong to dynamics).

5.2.1 HOMOGENEOUS TRANSFORMATION MATRICES

To represent **translation, rotation, and scaling** in a unified and computationally efficient way, VR systems use **4 × 4 homogeneous transformation matrices**.

General Form

$$T_{A \leftarrow B} = \begin{bmatrix} R_{3 \times 3} & P_{3 \times 1} \\ 0 & 1 \end{bmatrix}$$

Where:

- $R_{3 \times 3}$ is the **rotation matrix**
- $P_{3 \times 1}$ is the **translation (position) vector**
- The bottom row enables homogeneous coordinates

Interpretation

- The matrix expresses **coordinate system B relative to coordinate system A**
- Each **column of R** represents the projection of B's axes onto A's axes
- All coordinate systems are **orthonormal**:
 - Unit-length axes
 - Mutually perpendicular

Advantages of Homogeneous Matrices

1. Rotations and translations are treated uniformly
2. Multiple transformations can be **concatenated** using matrix multiplication
3. Inversion is straightforward:

$$T_{A \leftarrow B}^{-1} = \begin{bmatrix} R^T & -R^T P \\ 0 & 1 \end{bmatrix}$$

This is critical for real-time VR systems.

5.2.2 OBJECT POSITION

Object Coordinate System

Each virtual object is modeled in its **own local coordinate system**, typically:

- Origin at the **center of gravity**
- Axes aligned with **object symmetry**

Vertex coordinates (x, y, z) in this system **never change**, even when the object moves.

World Coordinate System

The virtual world has a **fixed reference frame**, called the **world coordinate system**.

To place an object into the world, we define a transformation:

$$T_{W \leftarrow i} = \begin{bmatrix} \mathbf{i}_{W \leftarrow i} & \mathbf{j}_{W \leftarrow i} & \mathbf{k}_{W \leftarrow i} & P_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where:

- $\mathbf{i}, \mathbf{j}, \mathbf{k}$ are the object axes expressed in world coordinates
- P_i is the position of the object origin in the world

This is shown in **Figure 5.15(a)** for a **static object**.

Time-Dependent Motion

When an object moves, the transformation becomes time-dependent:

$$T_{W \leftarrow i}(t)$$

Each vertex position in world coordinates is computed as:

$$V_i^{(W)}(t) = T_{W \leftarrow i}(t) V_i^{(i)}$$

This equation is applied to **all vertices** of the object.

Pure Translation Case (Figure 5.15b)

If the object only translates:

- Rotation matrix = identity
- Only the position vector changes

$$V_i^{(W)}(t) = \begin{bmatrix} 1 & 0 & 0 & p_x(t) \\ 0 & 1 & 0 & p_y(t) \\ 0 & 0 & 1 & p_z(t) \\ 0 & 0 & 0 & 1 \end{bmatrix} V_i^{(i)}$$

To move the object back, we multiply by the **inverse translation matrix**.

Scaling

Scaling is performed in **three steps**:

1. Translate object to world origin
2. Apply scaling
3. Translate object back

Scaling matrix:

$$T_{scaling} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Uniform scaling occurs when $s_x = s_y = s_z$.

5.2.3 TRANSFORMATION INVARIANTS

Concept of Invariance

A transformation is **invariant** if it does **not change with time**.

This concept is crucial in **interaction modeling**, especially for:

- Virtual hands
- Grasped objects
- HMD viewing

Example: Virtual Hand and Ball (Figure 5.16a)

- Tracker receiver position relative to tracker source:

$$T_{source \leftarrow receiver}(t)$$

- Source position in world:

$$T_{W \leftarrow source}$$

Hand vertex transformation:

$$V_i^{(W)}(t) = T_{W \leftarrow source} T_{source \leftarrow receiver}(t) V_i^{hand}$$

Relative Object Interaction

If a hand slaps or grasps a ball, we need **relative transformations**.

Relative hand-to-ball transformation:

$$T_{receiver \leftarrow ball}(t) = (T_{W \leftarrow receiver}(t))^{-1} T_{W \leftarrow ball}(t)$$

Grasping Case (Invariant Relationship)

Once the ball is grasped:

- Its position relative to the hand **does not change**
- $T_{receiver \leftarrow ball}$ becomes invariant

Ball position in world:

$$T_{W \leftarrow ball}(t) = T_{W \leftarrow source} T_{source \leftarrow receiver}(t) T_{receiver \leftarrow ball}$$

This greatly simplifies real-time computation.

5.2.4 OBJECT HIERARCHIES

Definition

Object hierarchies describe groups of objects that:

- Move together as a whole
- But whose parts can also move independently

This is modeled using **parent-child relationships**.

Mathematical Representation

If:

- Parent transformation = T_{parent}
- Child relative transformation = T_{child}

Then:

$$T_{world \leftarrow child} = T_{world \leftarrow parent} T_{parent \leftarrow child}$$

The **order of multiplication encodes the hierarchy**.

Tree Graph Representation

- Nodes $\rightarrow$ object segments
- Edges $\rightarrow$ transformations
- Parent node affects all children
- Children can move independently

Example: Football Player (Figure 5.17)

- Upper body → parent
- Arms and legs → children
- Lower arms → children of upper arms
- Lower legs → children of upper legs

Segmentation enables **realistic limb animation**.

Example: Virtual Hand (Figure 5.18)

Hierarchy:

- Palm → parent
- Fingers → children
- Finger links → sub-children

Fingertip position:

$$T_{W \leftarrow fingertip}(t) = T_{W \leftarrow source} T_{source \leftarrow receiver}(t) T_{receiver \leftarrow 1}(t) T_{1 \leftarrow 2}(t) T_{2 \leftarrow 3}(t) T_{3 \leftarrow fingertip}$$

Only the last matrix is invariant; others are time-dependent.

Including global motion:

$$T_{global \leftarrow fingertip}(t) = T_{global \leftarrow W}(t) T_{W \leftarrow fingertip}(t)$$

5.2.5 VIEWING THE THREE-DIMENSIONAL WORLD

View Transformation

The **view transformation** is the inverse of the global transformation:

$$T_{view} = (T_{global \leftarrow W})^{-1}$$

It places the camera in the world.

Camera Coordinate System

In OpenGL:

- Left-handed system
 - Camera looks along **positive z**
 - y axis points upward
 - x axis points right
-

Viewing Frustum (Figure 5.19a)

The camera sees only a **frustum**:

- Truncated pyramid
- Defined by:
 - left (l), right (r)
 - bottom (b), top (t)
 - near (n), far (f)

Perspective projection:

- Near objects appear larger
- Distant objects appear smaller

Canonical View Volume (Figure 5.19b)

To simplify processing, the frustum is mapped to a **unit cube**:

$$(-1, -1, -1) \rightarrow (1, 1, 1)$$

Projection matrix performs this mapping.

Projection Matrix

$$T_{projection} = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Clipping and Z-Buffering

- Objects outside the unit cube are **clipped**
- z values are normalized
- Z-buffer resolves visibility
- Only visible surfaces are rasterized

Screen Mapping

Final step:

- Map normalized (x, y) to screen pixels
- Scale and translate to window dimensions
- z coordinate is passed to Z-buffer unchanged

SUMMARY

Kinematics modeling provides:

- Precise object placement
- Time-dependent motion
- Hierarchical articulation
- Camera positioning
- Perspective viewing

It forms the **mathematical backbone of interactivity and realism** in virtual reality systems.

If you want next:

- A **short-answer exam version**
- A **numerical example**
- A **diagram-wise breakdown**
- Or a **direct mapping to OpenGL functions**

tell me what you want and I'll do it.

Below is a **complete, structured, and technically rigorous explanation of Section 5.3 – Physical Modeling**, written in **exam-ready Computer Graphics / VR theory style**, tightly aligned with **Figures 5.20 to 5.27** and the equations you provided.

No content is skipped, and all concepts are explained from first principles to implementation relevance.

5.3 PHYSICAL MODELING

After **geometric modeling** (shape) and **kinematics modeling** (position and motion), the final layer required for realism in a virtual environment is **physical modeling**.

Physical modeling introduces **material properties and interaction physics**, such as:

- Weight and inertia
- Surface hardness or softness (compliance)
- Surface roughness
- Deformation behavior (elastic or plastic)
- Contact forces

These properties allow the user not only to *see* objects but also to **feel** them through haptic interfaces.

All physical modeling computations are executed in the **haptics rendering pipeline**, which operates at much higher update rates than the graphics pipeline to ensure stable force feedback.

The stages of physical modeling follow the haptic pipeline:

1. Collision detection
2. Force computation
3. Force smoothing
4. Force mapping
5. Haptic texturing

5.3.1 COLLISION DETECTION

Collision detection determines **whether two or more objects are in contact**.

Only objects that collide are processed further by the haptics pipeline, making collision detection a form of **haptic clipping**.

Collision detection is classified into:

- **Approximate (bounding-box based)**
- **Exact (geometry-based)**

5.3.1.1 Approximate Collision Detection – Bounding Boxes

Definition

A **bounding box** is a rectangular prism that encloses all vertices of a 3D object.

Bounding boxes simplify collision tests by replacing complex geometry with simple volume overlap checks.

Types of Bounding Boxes (Figure 5.20a)

1. Oriented Bounding Box (OBB)

- Aligned with the object's principal axes
- Rotates with the object
- Tighter fit
- More accurate but computationally expensive

2. Axis-Aligned Bounding Box (AABB)

- Aligned with the **world coordinate axes**
- Does not rotate
- Faster to test
- Less accurate for rotated objects

3. Fixed-Size Bounding Box

- Special case of AABB
- Large enough to enclose the object in all orientations
- Fastest but least accurate

Bounding-Box Collision Test (Figure 5.20b)

Two objects collide if and only if their bounding boxes **overlap along all three world axes**:

- x-axis
- y-axis
- z-axis

For AABBs, this reduces to interval overlap tests on each axis, making them computationally efficient.

Performance Issues and Missed Collisions

Collision detection is performed **every frame by the CPU**.

In scenes with many objects or fast motion, collisions may be missed if:

$$d < \frac{v}{fps}$$

Where:

- d = maximum separation distance
- v = relative velocity
- fps = rendering frame rate

This is known as the **tunneling problem**, where objects pass through each other between frames.

Scene Coherence Optimization

Scene coherence means object positions change only slightly between frames.

Optimization strategy:

- Project all bounding boxes onto world axes
- Sort projection intervals once
- Update incrementally each frame

This reduces collision detection complexity to **O(n)** instead of **O(n²)**.

Multibody Collision Detection Pipeline (Figure 5.21)

1. Bounding-box pruning removes distant object pairs
2. Only potentially colliding pairs proceed
3. Exact collision detection is applied
4. Collision response is computed

This two-stage approach balances speed and accuracy.

5.3.1.2 Exact Collision Detection

Exact collision detection determines **where and how objects touch**, not just whether they overlap.

Feature-Based Collision Detection (Lin, 1993)

- Uses local geometric features:
 - Vertices
 - Edges
 - Facets
- Uses **Voronoi volumes** around features
- Collision occurs when a vertex enters the internal Voronoi region of another object

Performance is approximately **O(n)** per object, largely independent of polygon count.

Multibody Extension (Cohen et al., 1995)

- Extends Lin's method to many moving objects
- Reduces interaction checks from **O(n²)** to **O(n + m)**
where *m* is the number of nearby objects

Implicit Surface Collision Detection (Schlaroff & Pentland)

Objects are defined algebraically using implicit functions:

$$f(x, y, z) = 0$$

- $f(x) < 0$: point is inside object
- $f(x) > 0$: point is outside object

Collision occurs when a vertex satisfies $f(P) < 0$.

This method has complexity **O(m)**, where *m* is the number of tested points.

5.3.2 SURFACE DEFORMATION

After detecting collision, the system computes **collision response**.

If objects are non-rigid, this response includes **surface deformation**.

Surface deformation must be synchronized with the graphics pipeline because it changes object geometry in real time.

Parametric Surface Deformation

- Deformation applied indirectly via **control points**
- Difficult due to:
 - Occlusion
 - User unfamiliarity with spline control points

Direct Free-Form Deformation (Hsu et al.)

- User selects a surface point
- Moves it to a new position
- System computes control lattice changes using **least-squares optimization**

Polygonal Surface Deformation

- Deformation is performed directly by modifying vertex positions
- Neighboring vertices are updated using deformation propagation laws
- Supported by graphics libraries with mesh connectivity tables

Surface Cutting (Figure 5.22)

Surface cutting causes **topological changes**.

Process:

1. Closest vertex to cutting tool is duplicated (twin vertices)
2. Twins initially overlap
3. Twins are gradually separated along the cutting path
4. Process repeats to simulate continuous cutting

Used in:

- Surgical simulation
- Material fracture modeling

5.3.3 FORCE COMPUTATION

Force computation determines the **reaction forces felt by the user** through a haptic device.

Force models depend on:

- Contact type (point or multipoint)
- Object rigidity
- Deformation behavior
- Kinematics of interacting bodies

5.3.3.1 Elastic Virtual Objects

Elastic objects deform proportionally to applied force and return to original shape when released.

Hooke's Law Model

$$F_{constraint} = \begin{cases} K_{object} \cdot d & 0 < d < d_{max} \\ F_{max} & d_{max} < d \end{cases}$$

Where:

- K_{object} = stiffness
- d = penetration distance
- F_{max} = maximum device force

For elastic objects:

- Forces exist during both compression and relaxation
- Softer objects have lower stiffness and larger d_{max}

Non-Homogeneous Elastic Objects

Objects may have layers of different stiffness.

Piecewise force model:

- Soft exterior
- Hard interior
- Force slope increases after a threshold deformation d_{disc}

Virtual Pushbuttons (Figure 5.25)

Force profile simulates a tactile "click":

- Initial resistance from spring
- Sudden force drop (click)
- Rapid force increase at mechanical stop

Mathematically implemented using unit step functions.

5.3.3.2 Plastic Virtual Objects

Plastic objects **do not return to original shape** after deformation.

Initial Deformation

Initially behaves elastically:

$$F_{initial} = K_{elastic} \cdot d$$

During relaxation:

$$F = 0$$

Subsequent Deformations (Figure 5.26)

Force is zero until penetration exceeds previous deformation depth.

$$F_{subsequent} = \begin{cases} K_{plastic} \cdot d & 0 \leq d \leq d_{max} \\ F_{max} & d_{max} < d \\ 0 & \text{during relaxation} \end{cases}$$

This models materials like:

- Aluminum cans
 - Clay
 - Soft metals
-

5.3.3.3 Virtual Walls

Virtual walls are rigid, immobile surfaces.

Challenges:

- Real walls have extremely high stiffness
 - Haptic devices cannot generate such forces
-

Discrete-Time Instability (Figure 5.27)

Due to discrete control cycles:

- Forces are held constant per cycle
- Results in a **stair-step force profile**
- Causes energy injection and instability

Proportional-Derivative (PD) Wall Model

To stabilize interaction:

$$F_{constraint} = \begin{cases} K_{wall} \cdot d + B \cdot \dot{d} & \dot{d} < 0 \\ K_{wall} \cdot d & \dot{d} > 0 \end{cases}$$

Where:

- d = penetration depth
- $\dot{d}$ = penetration velocity
- B = damping coefficient

The damper:

- Acts only when moving **into** the wall
- Prevents oscillation
- Preserves clean release

Additional impulse terms may compensate for sensing and control delays.

SUMMARY OF PHYSICAL MODELING

Physical modeling provides:

- Realistic contact behavior
- Material differentiation
- Stable force feedback
- Interactive deformation and cutting

It integrates:

- Efficient collision detection
- Accurate force computation
- Stability control for haptics

Together, these enable **convincing, physically plausible interaction** in virtual reality systems.

If you want next, I can:

- Convert this into **short-answer exam format**
- Create **comparison tables (elastic vs plastic vs rigid)**
- Provide **numerical force-calculation examples**
- Link this directly to **haptic device control loops**

Just tell me what you need.

Below is a **complete, concept-by-concept explanation** of the material you posted, written in **clear academic / exam-oriented style**, covering **force smoothing, haptic texturing, behavior modeling, and model management**, with **direct linkage to Figures 5.28–5.33 and Equations (5.29)–(5.36)**. Nothing is skipped, and each idea is explained from **why it is needed** → **how it works** → **what it achieves in VR systems**.

5.3.4 FORCE SMOOTHING AND FORCE MAPPING

Problem: Faceted Feel of Polygonal Surfaces

In earlier force models, we assumed:

- Frictionless contact
- Single-point interaction
- Force direction aligned with the **surface normal**

This works well for **smooth analytical surfaces**, but **fails for polygonal meshes**.

A curved surface (like a cylinder or sphere) is approximated graphically by many flat polygons. While graphics shading hides this approximation visually, **haptics exposes it immediately**.

Why?

- Each polygon has a constant normal
- When the user crosses a polygon edge, the normal changes abruptly
- This causes sudden jumps in force direction
- The object feels **faceted or tessellated**, even if it looks smooth

Solution: Force Shading (Figure 5.28)

Just like **Gouraud/Phong shading** smooths lighting by interpolating normals, **force shading smooths haptic feedback** by interpolating force directions.

Definition

Force shading modifies the **direction** of the feedback force during contact with polygonal surfaces to simulate interaction with a smooth curved surface.

- **Force magnitude** remains unchanged
- **Only force direction is altered**

Understanding Figure 5.28

(a) Nonshaded contact forces

- Force is perpendicular to each polygon
- Direction changes abruptly at edges
- User feels sharp ridges

(b) Real cylinder contact forces

- Real curved surface produces **radial forces**
- Direction varies smoothly

(c) Shaded contact forces

- Polygon normals are interpolated
- Force direction approximates radial direction
- Haptic surface feels smooth

Mathematical Model (Equation 5.29)

$$F_{smoothed} = \begin{cases} K_{object} \cdot d \cdot \mathbf{N} & 0 < d < d_{max} \\ F_{max} \cdot \mathbf{N} & d_{max} < d \end{cases}$$

Where:

- K_{object} = object stiffness
- d = penetration depth
- F_{max} = maximum device force
- $\mathbf{N}$ = **interpolated normal vector**

HIP-Based Force Shading

For stylus-based devices (e.g., **PHANTOM**):

- Interaction is modeled at a single point called the **Haptic Interface Point (HIP)**
- Force shading is computed at the HIP
- Works best when:
 - Polygon count is high
 - Surface curvature is low

A coarse mesh cannot approximate curvature well, even with force shading.

Force Shading for Haptic Gloves (Figure 5.29)

Haptic gloves interact with surfaces using **entire fingertips**, not points.

Haptic Mesh Model

- Fingertip is modeled as a **cylindrical mesh**
- Multiple contact points exist simultaneously

At each mesh point i :

$$F_{mesh,i} = K_{ball} \cdot d_{mesh,i} \cdot \mathbf{N}_{surface,i}$$

Where:

- $d_{mesh,i}$ = penetration at mesh point i
- $\mathbf{N}_{surface,i}$ = weighted surface normal

This results in:

- More realistic force distribution
- Better perception of soft objects (e.g., squeezing a ball)

Mapping Mesh Forces to Real Gloves (Equation 5.31)

Commercial gloves (e.g., **CyberGrasp**) usually have:

- One actuator per finger

Therefore, all mesh forces are **summed and projected** onto actuator direction:

$$F_{displayed} = \sum F_{mesh} \cdot \cos(\theta)$$

Where:

- θ = angle between actuator direction and resultant force

5.3.5 HAPTIC TEXTURING

Haptic texturing adds **fine surface detail** beyond basic shape and stiffness.

Just as **graphics textures** modify appearance, **haptic textures** modify **touch sensation**.

They can convey:

- Roughness
- Slipperiness
- Stickiness
- Warmth/coldness (perceptually)
- Directional feel

Textures can also be **layered (cascaded)**, similar to multitexturing in graphics.

5.3.5.1 Textures Using Tactile (1-DOF) Mice (Figure 5.30)

Example: **iFeel Mouse**

Capabilities:

- Applies force only in **z-direction**
 - Cannot generate lateral friction
 - Surfaces are frictionless
-

Texture Parameters

- **Bump height** → force magnitude
- **Bump width** → duration of force pulse
- **Bump spacing** → texture frequency

Textures are **directional**:

- Different textures for +x and -x motion
- Enables simulation of **anisotropic materials**

Example:

- Velvet-like surfaces:
 - Smooth in one direction
 - Rough in the opposite direction

Grid textures (e.g., tennis racket strings) are created by symmetric parameters in both axes.

5.3.5.2 Textures Using the PHANToM

PHANToM supports:

- Full 3D force vectors
 - Friction
 - Viscosity
 - Inertia
-

Displacement-Map-Based Textures

Surface is defined by height function $h(x, y)$.

Texture forces depend on surface gradients:

$$F_{texture,x} = A \frac{\partial h}{\partial x}$$

$$F_{texture,y} = A \frac{\partial h}{\partial y}$$

This simulates:

- Hills and valleys
- Rough terrain
- Grooves and ridges

Vibratory Texture Model (Equation 5.34)

$$F_{texture} = A \sin(mx) \sin(ny)$$

Where:

- A = vibration amplitude
- m, n = frequency parameters
- $x, y \in (0, 2\pi)$

Interpretation:

- Large A , small $m, n \rightarrow$ perceivable shape
- Small A , large $m, n \rightarrow$ friction/roughness

Combined Force Model (Equation 5.35)

The total resistive force felt by the user is:

$$F_{resist} = F_{constraint} + F_{texture} + F_{friction}$$

This unified model allows:

- Shape perception
- Surface roughness
- Stickiness
- Material differentiation

5.4 BEHAVIOR MODELING

Until now, objects responded **only to user input**.

Behavior modeling allows objects to act **independently**.

Motivation

In large environments:

- User cannot control every object
- Objects must react to:
 - Time
 - Sensors
 - Other objects

Example:

- Sliding doors
- Clocks
- Thermometers
- Automatic lighting

Virtual Humans (Agents)

Definition

A **virtual human (agent)** is a 3D character capable of human-like behavior.

Groups of agents form **crowds**.

Levels of Autonomy (LOA)

According to Thalmann, each component can be:

1. Guided

- Requires user input or predefined paths

2. Programmed

- Executes scripted behavior

3. Autonomous

- Perceives environment and decides actions

Overall simulation autonomy:

$$S_{\text{autonomy}} = f(\text{LOA}(\text{objects}), \text{LOA}(\text{agents}), \text{LOA}(\text{groups}))$$

Agent Behavior Hierarchy

1. Reflex behavior

- Immediate responses

- Example: tackle opponent

2. **Emotion-based behavior**

- Filters perception through emotion
- Example: fear, aggression

3. **Cognitive behavior**

- Planning, memory, learning

Different agents may react differently to the same stimuli.

Examples

Dexter Agent (Figure 5.32a)

- Reflex-based handshake
- User controls interaction
- Agent responds with head movement

Mimicking Agent (Figure 5.32b)

- Recognizes user posture
 - Imitates gestures
 - Can switch to scripted dance routines
-

Crowd Behavior (Figure 5.33)

Crowds may:

- Follow leaders
- Share goals
- Exhibit memory

Example:

- Political demonstrations
- Building evacuation simulations

Crowd autonomy may differ from individual agent autonomy.

5.5 MODEL MANAGEMENT

Problem

Highly populated virtual worlds cause:

- High computation load
- Large memory usage
- Cache misses
- Disk swapping
- Frame rate drops
- Visual freezing

Need for Model Management

Model management ensures:

- Real-time performance
- Stable frame rates
- Scalable simulations

Techniques include:

- Level of Detail (LOD)
- Dynamic loading/unloading
- Spatial partitioning
- Behavioral simplification
- Multithreaded pipelines

Without proper management, even powerful hardware fails to maintain immersion.

FINAL SUMMARY

- **Force shading** removes haptic faceting by smoothing force direction
- **Haptic meshes** improve realism for multi-point contact
- **Haptic textures** add fine tactile detail using force modulation
- **Behavior modeling** introduces intelligent, autonomous objects and agents
- **Model management** ensures scalability and real-time performance

Together, these components transform a virtual environment from a static simulation into a **physically, perceptually, and behaviorally believable world**.

If you want, I can next:

- Convert this into **exam answers (5–10 marks)**
- Create **flow diagrams**
- Give **numerical problems**

- Relate this directly to **Unity / Unreal / OpenHaptics implementations**

Just tell me.